

Assignment 1: Data Structure Quest

Build it. Test it. Survive the edge cases.

Total: 100 XP (100 points)

YOUR MISSION

Complete the missing code in the provided stack and queue files. Your implementations must be array-based. The locations you need to complete are marked with // TODO.

COMPILER REQUIREMENT: Use GCC with C++17 only.

Quest Map

Level	Mission	XP
1	Build the Stack	40
2	Crack the Postfix Code	20
3	Master the Queue	40

Files You Will Use

- stack/myStack.cpp - complete the stack member functions.
- stack/myStack.h - provided stack class declaration.
- stack/stackTest.cpp - complete postfixTest() and use it to test your stack.
- queue/myQueue.h - complete the queue class template.
- queue/queueTest.cpp - provided test program for your queue.

LEVEL 1 | BUILD THE STACK

Create a reliable array-based integer stack.

**40
XP**

Mission: In myStack.cpp, implement the member functions of class myStack.

- Your stack must use an array-based implementation.
- Make push and pop work correctly.
- Correctly detect a full stack and an empty stack.
- Use stackTest.cpp to test your implementation.

LEVEL CHECK

Try filling the stack completely, pushing one extra value, emptying it completely, and popping one extra time. Your code should handle every case safely.

LEVEL 2 | CRACK THE POSTFIX CODE

Use your stack to evaluate postfix expressions.

20
XP

Mission: In `stackTest.cpp`, complete the function `postfixTest()`.

- Read the postfix expression one character at a time.
- Operands will be single-digit non-negative integers (0-9).
- Intermediate and final results may be negative or have multiple digits.
- Support the operators `+`, `-`, `*`, and `/`.
- Detect invalid or illegal postfix expressions.

Example challenge

Input: 4 5 7 * + 2 - =
Result: 37

Example trap

Input: 4 5 + -
Your program should detect that there are not enough operands.

LEVEL 3 | MASTER THE QUEUE

Build an array-based circular queue class template.

40
XP

Mission: In `myQueue.h`, implement the class template `myQueue`.

- The queue must be array-based.
- `arrayLength` must be one more than the maximum queue capacity.
- Make `currentSize()` calculate the number of elements correctly.
- Make `isFull()` and `isEmpty()` correctly identify the queue state.
- The number of stored elements must stay between 0 and `arrayLength - 1`.
- Test `enqueue` and `dequeue` carefully, including wrap-around behavior.

QUEUE PUZZLE

A circular queue can wrap from the end of the array back to the beginning. A queue that works only before wrap-around has not completed the level.

Boss Battle: Edge Cases

Matching the sample output is a good checkpoint, but it does not guarantee that your program is correct. Your solution may be tested with additional cases.

STACK	POSTFIX	QUEUE
Push when full	Too few operands	Enqueue when full
Pop when empty	Too many operands	Dequeue when empty
Fill and empty again	Negative intermediate result	Wrap around the array

Build Instructions

For the stack part, compile `myStack.cpp` and `stackTest.cpp` separately into `.o` files. Then, combine the two `.o` files to create the final executable program.

```
g++ -std=c++17 -c myStack.cpp
g++ -std=c++17 -c stackTest.cpp
g++ -std=c++17 myStack.o stackTest.o -o stackTest
```

For the queue part, you can compile queueTest.cpp directly because the queue template implementation is in myQueue.h.

```
g++ -std=c++17 queueTest.cpp -o queueTest
```

VS CODE USERS

You may run these commands in the VS Code terminal. VS Code is your editor; g++ still performs the compiling and linking.

Submission

- Place myStack.cpp, myStack.h, and stackTest.cpp inside a folder named stack.
- Place myQueue.h and queueTest.cpp inside a folder named queue.
- Zip the stack and queue folders together into one .zip file.
- Name the file a1_yourNetID.zip (example: a1_zz567.zip).
- Submit the .zip file to the Assignment 1 submission area in Canvas.

Final Quest Checklist

- All // TODO sections required by the assignment are completed.
- Stack handles both full and empty conditions.
- Postfix evaluator detects illegal expressions.
- Queue correctly handles full, empty, and wrap-around cases.
- Both test programs compile and run.
- Code is compiled using GCC with C++17 only.
- Folder structure and .zip filename are correct.

READY TO SUBMIT?

Do not rely only on the sample run. Try to break your own program before the Boss Battle does.